Go 语言笔试面试题(并发编程)

2020-09-05 23:10:10

golang interview questions

Go 语言笔试面试题汇总，Github

Q1 无缓冲的 channel 和 有缓冲的 channel 的区别？

答案

对于无缓冲的 channel，发送方将阻塞该信道，直到接收方从该信道接收到数据为止，而接收方也将阻塞该信道，直到发送方将数据发送到该信道中为止。

对于有缓存的 channel，发送方在没有空插槽（缓冲区使用完）的情况下阻塞，而接收方在信道为空的情况下阻塞。

例如:

```
func main() {
    st := time.Now()
    ch := make(chan bool)
    go func ()  {
        time.Sleep(time.Second * 2)
        <-ch
    }()
    ch <- true  // 无缓冲，发送方阻塞直到接收方接收到数据。
    fmt.Printf("cost %.1f s\n", time.Now().Sub(st).Seconds())
```

```go
    time.Sleep(time.Second * 5)
}

func main() {
    st := time.Now()
    ch := make(chan bool, 2)
    go func ()  {
        time.Sleep(time.Second * 2)
        <-ch
    }()
    ch <- true
    ch <- true // 缓冲区为 2，发送方不阻塞，继续往下执行
    fmt.Printf("cost %.1f s\n", time.Now().Sub(st).Seconds()) // cost 0.
0 s
    ch <- true // 缓冲区使用完，发送方阻塞，2s 后接收方接收到数据，释放一个
插槽，继续往下执行
    fmt.Printf("cost %.1f s\n", time.Now().Sub(st).Seconds()) // cost 2.
0 s
    time.Sleep(time.Second * 5)
}
```

Q2 什么是协程泄露(Goroutine Leak)？

答案

协程泄露是指协程创建后，长时间得不到释放，并且还在不断地创建新的协程，最终导致内存耗尽，程序崩溃。常见的导致协程泄露的场景有以下几种：

- 缺少接收器，导致发送阻塞

这个例子中，每执行一次 query，则启动 1000 个协程向信道 ch 发送数字 0，但只接收了一次，导致 999 个协程被阻塞，不能退出。

```go
func query() int {
    ch := make(chan int)
    for i := 0; i < 1000; i++ {
        go func() { ch <- 0 }()
    }
    return <-ch
}

func main() {
    for i := 0; i < 4; i++ {
        query()
        fmt.Printf("goroutines: %d\n", runtime.NumGoroutine())
    }
}
// goroutines: 1001
// goroutines: 2000
```

```
// goroutines: 2999
// goroutines: 3998
```

- 缺少发送器，导致接收阻塞

那同样的，如果启动 1000 个协程接收信道的信息，但信道并不会发送那么多次的信息，也会导致接收协程被阻塞，不能退出。

- 死锁(dead lock)

两个或两个以上的协程在执行过程中，由于竞争资源或者由于彼此通信而造成阻塞，这种情况下，也会导致协程被阻塞，不能退出。

- 无限循环(infinite loops)

这个例子中，为了避免网络等问题，采用了无限重试的方式，发送 HTTP 请求，直到获取到数据。那如果 HTTP 服务宕机，永远不可达，导致协程不能退出，发生泄漏。

```go
func request(url string, wg *sync.WaitGroup) {
    i := 0
    for {
        if _, err := http.Get(url); err == nil {
            // write to db
            break
        }
        i++
        if i >= 3 {
            break
        }
        time.Sleep(time.Second)
    }
    wg.Done()
}

func main() {
    var wg sync.WaitGroup
    for i := 0; i < 1000; i++ {
        wg.Add(1)
        go request(fmt.Sprintf("https://127.0.0.1:8080/%d", i), &wg)
    }
    wg.Wait()
}
```

Q3 Go 可以限制运行时操作系统线程的数量吗？

答案

The GOMAXPROCS variable limits the number of operating system threads that can execute user-level Go code simultaneously. There is no limit to the number of threads that can be blocked in system calls on behalf of Go code; those do not count against the GOMAXPROCS limit.

可以使用环境变量 GOMAXPROCS 或 runtime.GOMAXPROCS(num int) 设置，例如：

```
runtime.GOMAXPROCS(1) // 限制同时执行Go代码的操作系统线程数为 1
```

从官方文档的解释可以看到，GOMAXPROCS 限制的是同时执行用户态 Go 代码的操作系统线程的数量，但是对于被系统调用阻塞的线程数量是没有限制的。GOMAXPROCS 的默认值等于 CPU 的逻辑核数，同一时间，一个核只能绑定一个线程，然后运行被调度的协程。因此对于 CPU 密集型的任务，若该值过大，例如设置为 CPU 逻辑核数的 2 倍，会增加线程切换的开销，降低性能。对于 I/O 密集型应用，适当地调大该值，可以提高 I/O 吞吐率。